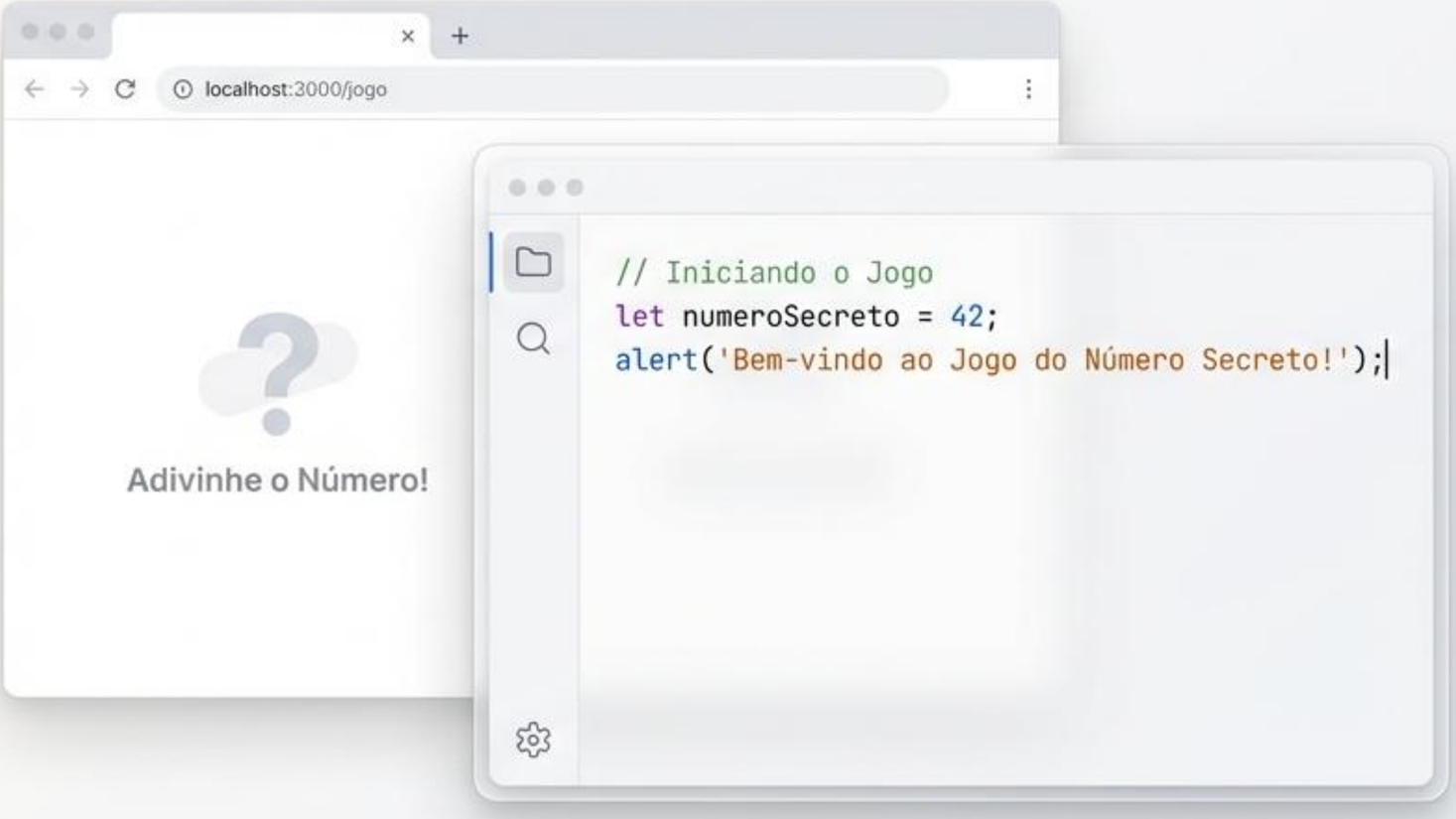
Lógica de Programação com JavaScript

Construindo o Jogo do Número Secreto



Desenvolvimento Front-End II | 3º Ano Técnico

NotebookLM

Executa a função

```
alert('Mensagem aqui');
```

Texto (String)

Saída de Dados: A função alert()

O comando alert exibe uma caixa de diálogo simples. Atenção aos parênteses e aspas.

NotebookLM

Aplicando no Projeto

```
alert('Boas-vindas ao jogo do Número Secreto');
```

NotebookLM

Armazenamento e Entrada

```
let nomeDaVariavel = valor;
let entrada = prompt('Sua pergunta aqui');
```

NotebookLM

Definindo o Cenário

```
1  let numeroSecreto = 29;
2  let chute = prompt('Escolha um número entre 1 e 30');
```

NotebookLM

Lógica de Decisão: if

Igualdade (Comparação)

```
if (condicao == verdadeira) { ... }
```

NotebookLM

Validando a Vitória

```
1   if (chute == numeroSecreto) {
2     console.log('Isso aí! Você descobriu o número secreto (29)');
3   }
```

NotebookLM

Fluxo Alternativo: else

```
// Verifica se acertou
if (...) { ... } else { ... }
```

// Comentários ajudam a documentar a lógica

NotebookLM

Feedback de Erro

```
1 if (chute == numeroSecreto) {
2   alert('Você descobriu!');
3 } else {
4   alert('Você errou :(');
5 }
```

NotebookLM

Texto Dinâmico: Template Strings

Concatenação (Antigo)

```
1  'O número é ' + numero
```

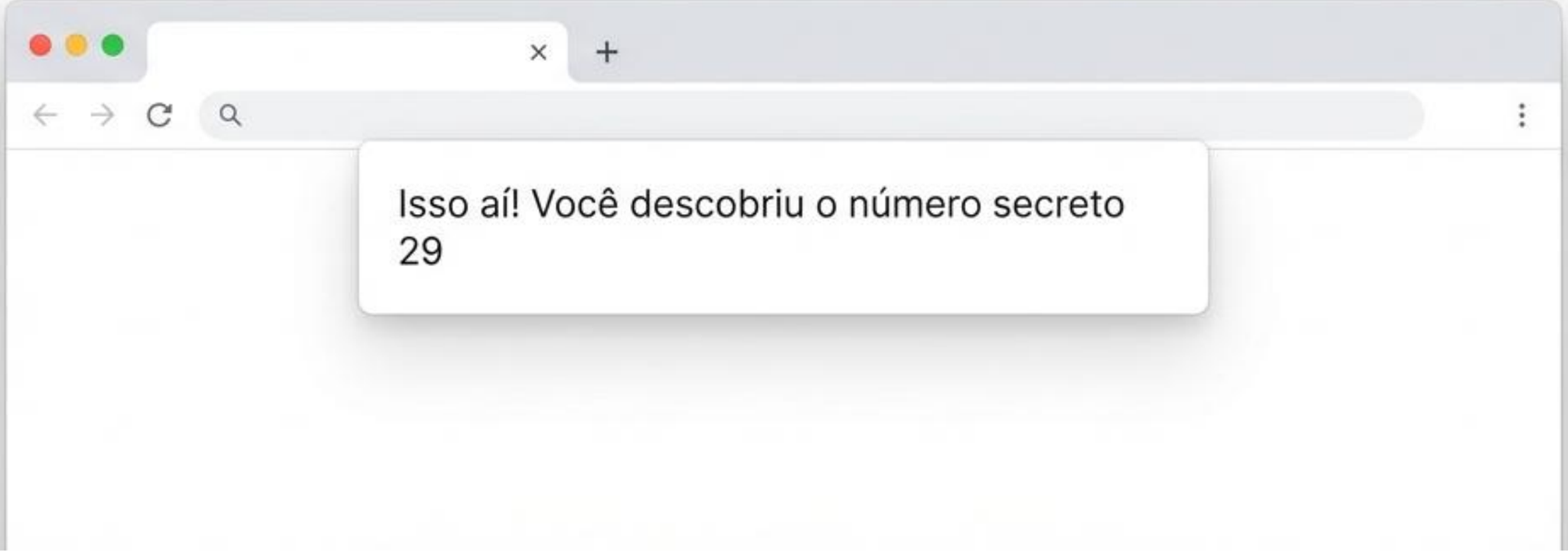
Template String (Novo)

```
1  `O número é ${numero}`
```

NotebookLM

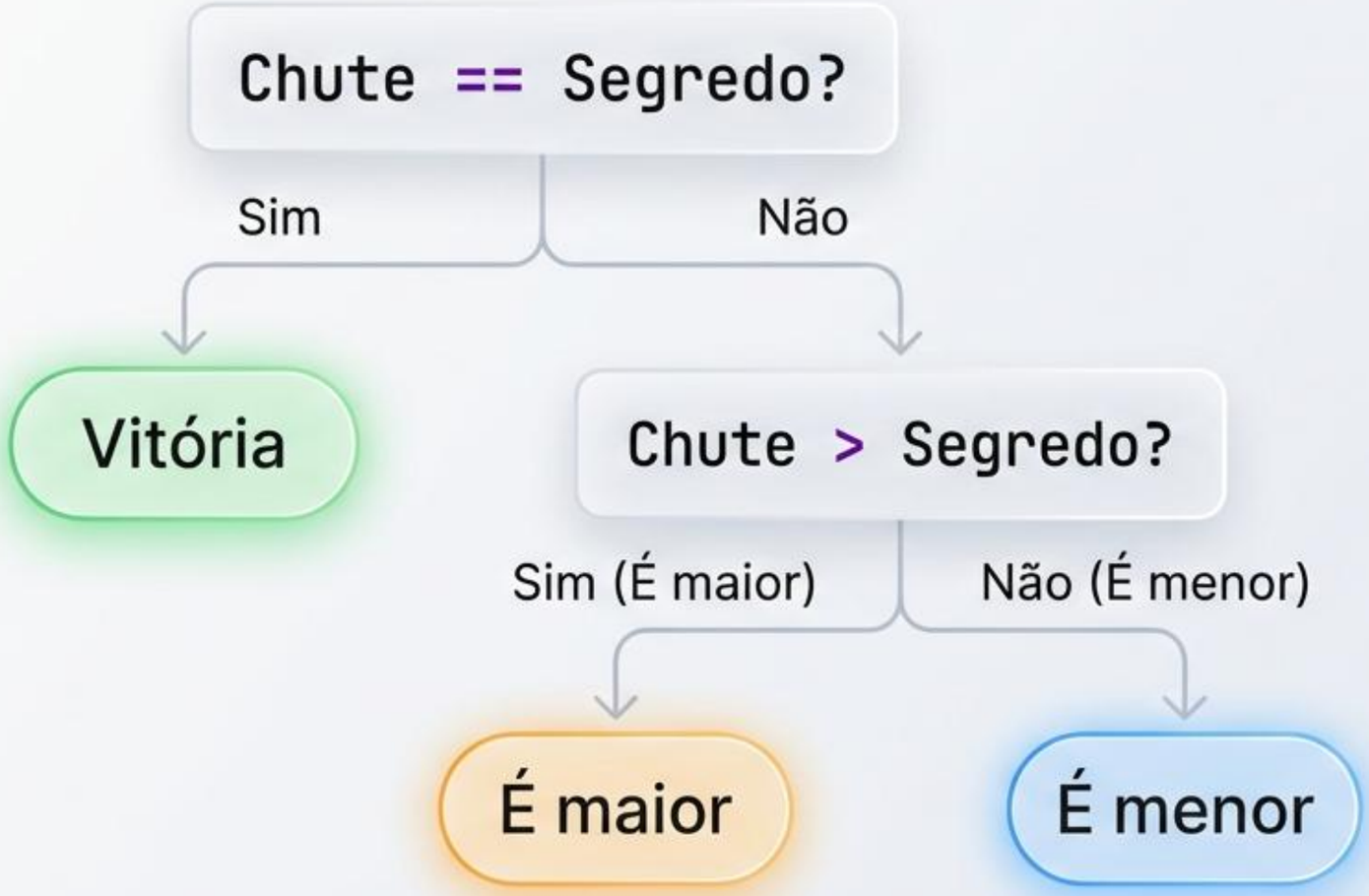
Revelando o Número Secreto

```
1  alert(`Isso aí! Você descobriu o número secreto ${numeroSecreto}`);
```



NotebookLM

Condicionais Aninhadas



>

Maior que

<

Menor que

NotebookLM

Sistema de Dicas

```
1  } else {
2    if (chute > numeroSecreto) {
3      alert('O número secreto é menor que ${chute}')
4    } else {
5      alert('O número secreto é maior que ${chute}')
6    }
7  }
```

NotebookLM

Estruturas de Repetição: while

Executa enquanto a condição for verdadeira

Diferente de (Não igual)

```
while (chute != numeroSecreto) {
    ...
}
```

NotebookLM

O Loop do Jogo

```
1   alert('Boas vindas ao jogo do número secreto');
2   let numeroSecreto = 5;
3   console.log(numeroSecreto);
4   let chute;
5
6   // enquanto chute não for igual ao n.s.
7   while (chute != numeroSecreto) {
8       chute = prompt('Escolha um número entre 1 e 100');
9       // se chute for igual ao número secreto
10      if (chute == numeroSecreto) {
11          alert(`Isso aí! Você descobriu o número secreto ${numeroSecreto}`);
12      } else {
13          if (chute > numeroSecreto) {
14              alert(`O número secreto é menor que ${chute}`);
15          } else {
16              alert(`O número secreto é maior que ${chute}`);
17          }
18      }
19  }
```

NotebookLM

Contadores e Incremento

Atalho (Shorthand)

```
tentativas = tentativas + 1
```

NotebookLM

Implementando o Score

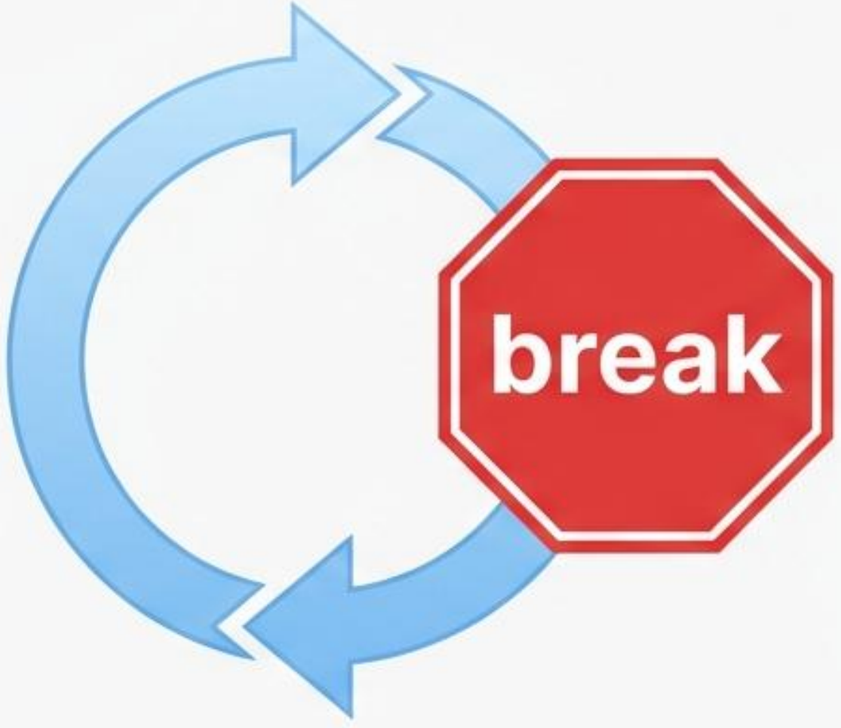
```
1   alert('Boas vindas ao jogo do número secreto');
2   let numeroSecreto = 5;
3   let tentativas = 1;
4   console.log(numeroSecreto);
5   let chute;
6
7   // enquanto chute não for igual ao n.s.
8   while (chute != numeroSecreto) {
9       chute = prompt('Escolha um número entre 1 e 100');
10      // se chute for igual ao número secreto
11      if (chute == numeroSecreto) {
12          alert(`Isso aí! Você descobriu o número secreto ${numeroSecreto} com ${tentativas} tentativas.`);
13      } else {
14          if (chute > numeroSecreto) {
15              alert(`O número secreto é menor que ${chute}`);
16          } else {
17              alert(`O número secreto é maior que ${chute}`);
18          }
19          tentativas++;
20      }
21  }
```

Inicializar

Incrementar

NotebookLM

Interrupção de Loops




```
if (chute == numeroSecreto) {
  break;
}
```

NotebookLM

Operador Ternário (if/else em uma linha)

Variável = **Condição** ? **Valor Verdadeiro** : **Valor Falso**

```
let resultado = (pontos > 10) ? 'Vencedor' : 'Perdedor';
```

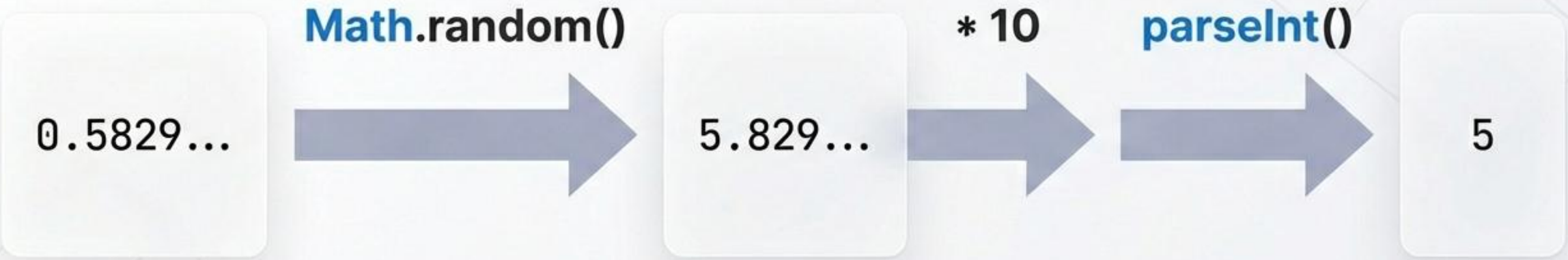
NotebookLM

Ajuste Gramatical

```
let palavraTentativa = tentativas > 1 ? 'tentativas' : 'tentativa';
```

NotebookLM

Gerando Números Inteiros




```
let numeroAleatorio = parseInt(Math.random() * 10);
```

NotebookLM

Estrutura de Controle: Condicionais em JavaScript

GUIA DE SINTAXE (O CÓDIGO)

```
if (condição) {
    ...
} else {
    ...
}
```

Bloco Verdadeiro

Bloco Falso

Estrutura if e else

Define o bloco de código executado quando uma condição é verdadeira ou falsa.

```
if (condição1) {
    ...
} else {
    ...
    if (condição2) {
        ...
    } else {
        ...
    }
}
```

Condicionais Aninhadas

Uso de um if dentro de um else para múltiplas verificações lógicas.

Template Strings (Interpolação)

Uso de crases e **$(variável)** para inserir valores dinâmicos dentro de textos.

```
`Chute o número é é ${variável}`
```

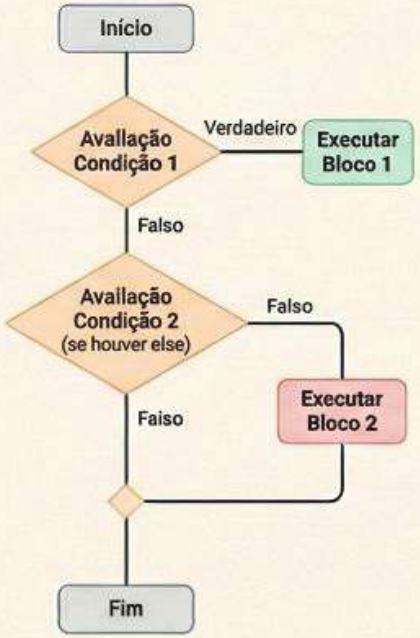
Operadores de Comparação Fundamentais

Operador	Significado	Exemplo de Uso
==	Igualdade	chute == numeroSecreto
>	Maior que	chute > numeroSecreto
<	Menor que	numeroSecreto < chute

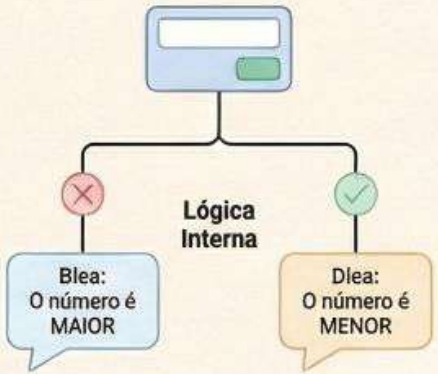
FUNCIONAMENTO E LÓGICA (O ENTENDIMENTO)

Fluxo de Decisão Linear

O JavaScript executa o código linha por linha, avaliando cada condição sequencialmente.



Verificação de Erro e Dicas



Ao errar, a lógica interna decide se a dica será "maior" ou "menor".

Validação de Entrada

Palpite do Usuário **(chute)**

Valor Armazenado na Memória **(numeroSecreto)**

O programa compara o palpite **(chute)** com o valor armazenado na memória.

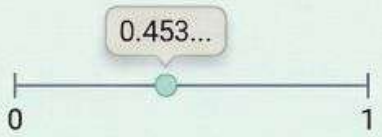
NotebookLM

Dominando Números Aleatórios em JavaScript

DICIONÁRIO DE SINTAXE

Math.random()

- Função que gera um número decimal aleatório entre 0 (inclusivo) e 1 (exclusivo).



parseInt()

- Método que analisa um argumento e retorna um número inteiro, descartando os valores decimais.

Declaração Completa

```
let numeroSecreto = parseInt(Math.random() * 10 + 1);
```

EXPLICAÇÃO E LÓGICA

Definindo o Intervalo Máximo

- Multiplicar o resultado por 10 ou 100 desloca a vírgula para definir o limite.

Deslocamento de Valor (Início)

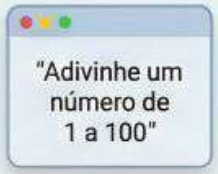
- Somar "+ 1" ao final garante que o número 0 seja excluído e o limite máximo incluído.

Exemplos de Alcance do Número Secreto

Multiplicador	Intervalo Resultante (com +1)	Exemplo de Uso
10	1 a 10	Jogos rápidos
100	1 a 100	Jogos de maior dificuldade

O Conceito de Acoplamento

Ao alterar o intervalo maternático, deve-se atualizar também a instrução de texto (prompt) ao usuário.



NotebookLM

Guia Prático: O Operador Ternário no JavaScript

ÁREA 1: SINTAXE DA LINGUAGEM

Estrutura de Três Partes

EXPRESSÃO SE VERDADEIRO

CONDIÇÃO

```
condição ? expressão_se_verdadeiro : expressão_se_falso ;
```

AVALIAÇÃO

EXPRESSÃO SE FALSO

SEPARAÇÃO

Atribuição com Ternário

```
let palavraTentativa = tentativas > 1 ? 'tentativas' : 'tentativa';
```

Símbolos de Controle

- **?** O ponto de interrogação inicia a verificação
- **:** os dois-pontos separam o resultado verdadeiro do falso.

ÁREA 2: LÓGICA E FUNCIONAMENTO

Refatoração de if...else

Comparação: Tradicional vs. Ternário

Estrutura Tradicional (if...else)

```
if (tentativas > 1) {
  palavra = 'tentativas'
}
} else {
  palavra = 'tentativa'
}
```

Estrutura com Operador Ternário

```
palavra = tentativas > 1 ?
  'tentativas' : 'tentativa';
```

Substitui blocos de 4 ou mais linhas por apenas uma, mantendo o mesmo comportamento funcional.

O Fluxo de Decisão

POSITIVA: Executa o que vem após o '?'

NEGATIVA: Executa o que vem após o ':'

Mentalidade de Leitura

Leia como: "Esta condição é verdadeira? Se sim, use isto; caso contrário, use aquilo."

NotebookLM

JavaScript: A Sintaxe e o Funcionamento do Comando alert()

GUIA DO COMANDO ALERT()

SINTAXE E ESTRUTURA DA LINGUAGEM

app.js

```
alert('Olá, mundo!');
```

A FUNÇÃO alert()

`alert()`

Escrita sempre em minúsculo e seguida obrigatoriamente por parênteses que envolvem o conteúdo.

USO DE ASPAS (STRINGS)

```
alert('Texto com aspas simples');
alert("Texto com aspas duplas");
```

Mensagens de texto devem estar entre aspas simples ou duplas, mantendo a padronização.

FINALIZAÇÃO COM PONTO E VÍRGULA

`;`

O caractere ";" indica o término de um comando, sendo essencial para a organização.

FUNCIONAMENTO E FLUXO DE EXECUÇÃO

app.js

O PAPEL DO ARQUIVO app.js

É o local exclusivo para a lógica de programação, separado do HTML e CSS.

INTERAÇÃO COM O USUÁRIO

O comando interrompe a navegação para exibir um **pop-up** com a mensagem definida.

Olá, mundo!

RESUMO DOS ARQUIVOS FUNDAMENTAIS

Arquivo	Função Principal
index.html	Estrutura visual e interface do robô
style.css	Definições de estilo e aparência
app.js	Inteligência e comandos JavaScript

CICLO SALVAR E ATUALIZAR

Ctrl+S + F5

Utilize **Ctrl+S** para salvar o código e **F5** no navegador para testar alterações.

NotebookLM

Guia de Sintaxe: Variáveis e Comandos de Interação em JavaScript

Sintaxe - Código e Comandos

```
alert('mensagem');
```

Saída de Dados

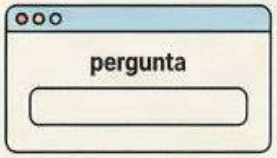
Comando utilizado para exibir uma janeia de alerta com uma mensagem para o usuário.

```
let nomeDaVariavel = valor;
```

Declaração de Variáveis

Palavra-chave para criar variáveis e atribuir valores que serão armazenados na memória.

```
prompt('pergunta');
```



Entrada de Dados

Exibe uma caixa de diálogo com um campo de texto para entrada de dados.

Explicação - Lógica e Funcionamento

Variáveis e Memória

Declarar uma variável com "let" reserva um espaço na memória para guardar informações reutilizáveis.

Padrão CamelCase

Nomes de variáveis devem ser claros, sem acento, iniciando a segunda palavra em maiúscula.

✓ `nomeDaVariavel`

Claro, sem acentos, segunda palavra malóscula.

✗ `nome_da_variavel`

✗ `nomedavariavel`

Evitar

Strings vs. Números e Atribuição

Tipo de Dado	Exempio de Sintaxe	Observação Visual
T Texto (String)	`let mensagem = 'Olá';`	`'Olá'` Utiliza aspas simples
# Numérico (Number)	`let numero = 29;`	`29` Sem aspas
Entrada (Input)	`let chute = prompt('..');`	Armazena o que o usuário digitar

NotebookLM

Dominando Condicionais e Comentários em JavaScript

Dicionário de Sintaxe

Comentários de Linha Única (`//`)

```
//
```

Textos após `//` são ignorados pelo interpretador, servindo apenas para documentação humana.

Estrutura `if` (Se)

```
if (condição) {
  bloco
}
```

Define o código que será executado apenas se a condição for verdadeira.

Estrutura `else` (Senão)

```
else {
  bloco
}
```

Define a ação alternativa executada obrigatoriamente quando a condição do `if` é falsa.

Lógica e Funcionamento

Fluxo de Execução Linear

O JavaScript lê o código linha por linha, executando instruções na ordem em que aparecem.

Console.log vs. Alert

console.log

>_ Log: Depuração (desenvolvedor)

alert

Mensagem ao Usuário

Use `console.log` para depuração e `alert` para mensagens diretas ao usuário final.

Variáveis vs. Strings (Aspas)

Nomes sem aspas acessam valores de variáveis.

Nomes entre aspas são interpretados apenas como texto.

Categorias de Comandos

Comando	Categoria	Objetivo
`if / else`	Fluxo	Controlar decisões do programa
`/`	Documentação	Explicar trechos do código
`console.log()`	Saída	Visualizar dados internamente

NotebookLM

Estrutura de Repetição: O Laço `while` em JavaScript

Área 1: Sintaxe e Operadores

Estrutura Básica do `while`

```
while (condição) {
  // instruções a serem executadas
}
```

Operadores de Comparação Comuns

!=
Não é igual / Diferente
`chute != numeroSecreto`

==
Igualdade
`chute == numeroSecreto`

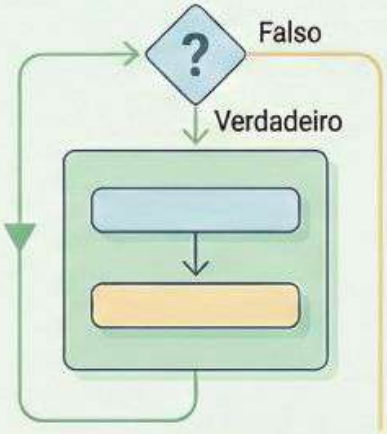
>
Maior que
`chute > numeroSecreto`

Importância da Indentação

O recuo à esquerda (espaços/tab) dentro das chaves garante a legibilidade do escopo.

Área 2: Lógica e Funcionamento

Execução Condicional

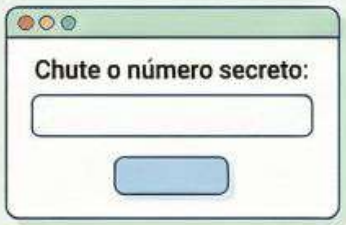


O código interno só é executado se a condição inicial for verdadeira.

O Risco do Loop Infinito

Se a condição nunca se tornar falsa, o programa travará em repetição perpétua.

Aplicação Prática (Loop de Jogo)



Repete a pergunta de chute ao usuário até que ele acerte o número secreto.

NotebookLM

```
1   let numeroSecreto = parseInt(Math.random() * 11)
2   let tentativas = 1
3   let chute
4
5   alert('Boas vindas ao jogo do número secreto')
6
7   while (chute != numeroSecreto) {
8
9       let chute = prompt('Escolha um número entre 1 e 10')
10
11      if (chute == numeroSecreto) {
12          break
13      } else {
14          if (chute > numeroSecreto) {
15              alert(`O número secreto é menor que ${chuve}`);
16          } else {
17              alert(`O número secreto é maior ${chuve}`);
18          }
19      }
20      tentativas++
21  }
22
23  let palavraTentativa = tentativas > 1 ? 'tentativas' : 'tentativa'
24  alert(`O número secreto era ${numeroSecreto} e você acertou com apenas ${tentativas}
    ${palavraTentativa}`)
```